

Ottimizzazione con insiemi È possibile migliorare significativamente la funzione `isPaired()` usando un insieme basato su tabella hash per memorizzare i caratteri della sottostringa. Per ciascun carattere c presente, verifichiamo in tempo costante se è presente anche `swapcase(c)`.

```
boolean isPaired(item[] S, int start, int end)
```

```
    SET letters = Set()
    for i = start to end do
        | letters.insert(S[i])
    foreach c ∈ letters do
        | if not letters.contains(swapcase(c)) then
            | | return false
    return true
```

Questa ottimizzazione riduce il costo della verifica da $\Theta(n^2)$ a $\Theta(n)$, portando la complessità complessiva dell'algoritmo da $\Theta(n^4)$ a $\Theta(n^3)$.

Approccio divide-et-impera Per ottenere una soluzione più efficiente, osserviamo che se un carattere c compare nella sottostringa $S[i \dots j]$ ma `swapcase(c)` non compare, allora *nessuna sottostringa che lo contiene può essere pareggiata*. Possiamo quindi dividere il problema in due sottoproblemi escludendo c .

```
int largestPaired(item[] S, int n)
```

```
    return lpRec(S, 1, n)
```

```
int lpRec(item[] S, int i, int j)
```

```
    if i > j then
        | return 0
    HASH letters = Hash()
    for k = i to j do
        | letters[S[k]] = k
    foreach c ∈ letters do
        | if not letters.contains(swapcase(c)) then
            | | int m = letters[c]
            | | return max(lpRec(S, i, m - 1), lpRec(S, m + 1, j))
    return j - i + 1
```

Nel caso peggiore (ogni carattere è sbilanciato), l'algoritmo esegue una divisione di un carattere per volta, e il costo è:

$$T(n) = T(n - 1) + n = \Theta(n^2)$$

Nel caso migliore (la stringa è interamente pareggiata), la funzione restituisce subito la sua lunghezza, con costo $\Omega(n)$. Quindi la complessità complessiva è compresa tra $\Omega(n)$ e $\Theta(n^2)$.

Esercizio B1 – Braccia rubate alla ricerca – Punti ≥ 8

Idea generale. Il problema può essere modellato come un problema di *flusso massimo* in un grafo bipartito, in cui colleghiamo ogni persona agli uffici che è disposta a pulire, rispettando il vincolo massimo m sul numero di uffici assegnabili a ciascuna persona.

Costruzione del grafo. Costruiamo un grafo orientato $G = (V, E)$ con i seguenti nodi e archi:

- un nodo sorgente s ;
- un nodo pozzo t ;
- un nodo per ogni persona $\ell \in L$;
- un nodo per ogni ufficio $o \in O$;

Colleghiamo i nodi come segue:

- da s a ogni persona $\ell \in L$, un arco con capacità m ;

- da ogni persona ℓ a ogni ufficio $o \in A_\ell$, un arco con capacità 1;
- da ogni ufficio $o \in O$ a t , un arco con capacità 1.

Calcoliamo il flusso massimo da s a t . Se il valore del flusso è pari a $|O|$, allora esiste una pianificazione valida: possiamo assegnare ogni ufficio a una persona in modo conforme ai vincoli. Altrimenti, non esiste una pianificazione che soddisfa tutti i requisiti.

- Ogni ufficio può essere pulito una sola volta perché l'arco verso t ha capacità 1;
- ogni persona può pulire al massimo m uffici perché l'arco da s alla persona ha capacità m ;
- il vincolo A_ℓ è rispettato perché colleghiamo ogni persona solo agli uffici da lei dichiarati;
- il flusso costringe ogni assegnazione a essere uno-a-uno tra persona e ufficio.

Sia $n_L = |L|$, $n_O = |O|$. Il numero totale di nodi è $n_L + n_O + 2$, il numero di archi è $O(n_k \cdot n_O) + n_L + n_O$. Il limite superiore al flusso è dato dal numero di uffici, $f^* \leq n_O$. Usando l'algoritmo di Ford-Fulkerson, la complessità è $O(|f^*|(V + E)) = O(n_O(n_L \cdot n_O + n_L + n_O)) = O(n_L \cdot n_O^2)$.

Esercizio B1 – Dollar words – Punti ≥ 8

L'esercizio è simile a quello proposto nel compito del 17/01/2023 (utilizzato come compito di prova), che a sua volta è una generalizzazione dell'Esercizio 4 del 12/01/2015 (modi di scendere una scala). Nel compito del 2015 si ottenevano i numeri di Tetranacci; in quello del 2023, si ottenevano i numeri di Fibonacci di ordine s . In questo problema, si ottengono i numeri di ordine 26, detti anche *numeri di Hexacosanacci*!

Non bisogna farsi ingannare dal fatto che i primi numeri della sequenza sono potenze di 2: dopo il 26-esimo termine, questo non è più vero, poiché non si possono più aggiungere lettere con valore 27 o superiore. Il 27-esimo valore è quindi 67108863, non 67108864 (che è 2^{26}), e da quel punto in poi i numeri di Hexacosanacci crescono più lentamente delle potenze di 2.

Per risolvere il problema, utilizziamo la *programmazione dinamica*. Sia $DP[t]$ il numero di stringhe (di qualunque lunghezza, con lettere ripetute e ordine rilevante) che hanno valore complessivo pari a t centesimi.

La formula ricorsiva è la seguente:

$$DP[t] = \begin{cases} 1 & \text{se } t = 0 \\ \sum_{i=1}^{\min(26, t)} DP[t-i] & \text{se } t > 0 \end{cases}$$

Nel caso base $t = 0$, esiste una sola stringa: la stringa vuota. Per $t > 0$, si considera ciascun possibile valore di lettera i compreso tra 1 e $\min(26, t)$ (corrispondente alle lettere "a" fino a quella di valore massimo compatibile con t), e lo si concatena a tutte le stringhe che valgono esattamente $t - i$.

In questo modo, costruiamo tutte le stringhe di valore t come l'unione di una lettera finale e di una stringa che vale $t - i$.

L'algoritmo risultante è il seguente:

```

int countWords(int T)


---


    int[] DP = new int[0...T]
    DP[0] = 1
    for  $t = 1$  to  $T$  do
        DP[t] = 0
        for  $i = 1$  to  $\min(26, t)$  do
            DP[t] = DP[t] + DP[t - i]
    return DP[T]

```

La complessità dell'algoritmo è $\Theta(T)$, poiché l'iterazione esterna ha T passi e quella interna è limitata da una costante (26), corrispondente al numero di lettere dell'alfabeto inglese.

Esercizio B3 – Tuple ben ordinate – Punti ≥ 12

L'algoritmo utilizza un approccio basato su backtracking per generare tutte le *tuple ben ordinate* di lunghezza compresa tra 1 e k , estraendo i valori da un vettore $A[1 \dots n]$.

La procedura principale `printTuples()` inizializza un array ausiliario $S[1 \dots k]$ che viene utilizzato per costruire ricorsivamente le tuple. Essa invoca due procedure ricorsive distinte:

- `printRecUp()` genera tutte le tuple strettamente crescenti. A ogni passo, la funzione esplora i possibili valori successivi di A , aggiungendo un nuovo elemento alla tupla corrente solo se è strettamente maggiore dell'ultimo valore inserito. Per ogni nuova tupla parziale costruita, ne stampa immediatamente una copia.
- `printRecDown()` genera simmetricamente tutte le tuple strettamente decrescenti, imponendo che ogni nuovo valore aggiunto sia strettamente minore dell'ultimo.

Entrambe le funzioni esplorano tutte le combinazioni possibili mediante una ricorsione su due parametri:

- i rappresenta la posizione corrente nella tupla parziale;
- j rappresenta l'indice corrente nel vettore A da cui considerare i prossimi elementi.

In ciascun passo ricorsivo, il controllo $j \leq n$ assicura che non si ecceda la lunghezza di A , mentre $i \leq k$ impedisce che le tuple superino la dimensione desiderata. Le condizioni di confronto " $<$ " per la versione crescente, " $>$ " per quella decrescente) garantiscono il rispetto del vincolo di ordinamento.

Il caso peggiore avviene quando $k = n$ e il vettore A è ordinato in maniera crescente o decrescente, nel qual caso vengono generati tutti i sottoinsiemi, che sono 2^n ; se il vettore non è ordinato, il meccanismo di pruning evita di stampare elementi che non siano ordinati. La complessità è quindi $O(n \cdot 2^n)$, che include il fattore $\Theta(n)$ per la stampa. Ho dato 10% in meno a chi ha generato tutti i possibili sottovettori, per poi verificare se rispetta le condizioni di essere una tupla ben ordinata, che ha costo $\Theta(n \cdot 2^n)$.

```
printTuples(int A[], int n, int k)
```

```
    int[] S = new int[1...k]
    printRecUp(A, n, k, S, 1, 1)
    printRecDown(A, n, k, S, 1, 1)
```

```
printRecUp(int A[], int n, int k, int S[], int i, int j)
```

```
    if i ≤ k and j ≤ n then
        printRecUp(A, n, k, S, i, j + 1)
        if i = 1 or S[i - 1] < A[j] then
            S[i] = A[j]
            if i > 1 then
                print S[1...i]
            printRecUp(A, n, k, S, i + 1, j + 1)
```

```
printRecDown(int A[], int n, int k, int S[], int i, int j)
```

```
    if i ≤ k and j ≤ n then
        printRecDown(A, n, k, S, i, j + 1)
        if i = 1 or S[i - 1] > A[j] then
            S[i] = A[j]
            if i > 1 then
                print S[1...i]
            printRecDown(A, n, k, S, i + 1, j + 1)
```
